Build an Alert Triage Agent with LangGraph

Colin Vitols | IT7075: Applied AI for Cybersecurity | November 10, 2026

Project folder on GitHub

Video: pending - the required 5-10 minute narrated recording has not yet been supplied.

Concept check

1. What is state? In my graph, state is the shared record carrying the alert, event metadata, risk, context, and final decision. Each node reads the fields it needs and returns only its changes, so the next node can use the updated record. [1]

2. What are nodes, edges, and END? A node is a Python function that performs one step, such as classifying an alert or checking context. An edge connects those steps, and END finishes that invocation after investigate or quick_close has written the report. [1]

3. How does the graph choose? My conditional edge calls route_by_risk and maps its returned label to investigate or quick_close. For A06, the baseline takes quick_close, but the added context node changes the risk to high and sends the same alert to investigation. [1]

4. What is a tool? A tool is a function a node calls to obtain information or perform a task. My lookup_context tool reads simulated asset and change records, providing context that a text-only keyword check does not have; it does not contact a real service.

5. What does checkpointing provide? The checkpointer saves graph state under a thread_id, so a later call for the same incident can retain its event details. My output also shows that another thread stays separate; this in-memory implementation loses its checkpoints when the Python process exits. [2]

Connection to my earlier work

My earlier projects retrieved incident-triage guidance with manual RAG and then LangChain. Here I kept the triage setting but changed the task: the workflow decides whether an analyst should investigate an alert. Every project node runs ordinary Python, so these are exact routing results rather than generated answers.

Prepared with AI assistance. The synthetic labels, explanations, and notebook should be reviewed before presenting the live walkthrough.

What I built

The baseline is classify -> conditional edge -> investigate or quick_close -> END. G1 adds context_review between classify and that conditional edge. The new node calls a read-only fixture lookup, then checks production audit shutdowns and narrowly matched maintenance or lab-test approvals.

Approval requires the actor, asset, action, and timezone-aware time to match a separate record. An unapproved StopLogging event on a production audit asset becomes high risk. A verified group change or verified lab test can become low risk; otherwise, the keyword decision stands. Ground-truth labels and notes never enter graph state.

The twelve scenarios

ID	Label	Scenario and reason
A01	Escalate	RCE payload plus a web-process shell: correlated execution evidence.
A02	Escalate	Ransomware behavior with mass unreadable files: active impact.
A03	Escalate	Unapproved Domain Admins addition: unexpected privilege.
A04	Escalate	Credential dumping through an unsigned LSASS reader.
A05	Escalate	Outbound payroll archives to an unexplained destination.
A06	Escalate	StopLogging disables the production audit trail without a ticket.
A07	Close	Administrator group change matches the approval and window.
A08	Close	Critical test signature matches the approved lab exercise.
A09	Close	Outbound backup copy matches the established job and checksums.
A10	Close	User-requested password reset after identity verification.
A11	Close	Single failed logon followed by the same device/user retry.
A12	Close	Normal sensor-health heartbeat, with collectors online.

Why the hard cases have those labels

A06 contains none of the words used in any experiment, yet stopping production audit collection creates an immediate visibility gap. AWS documents that StopLogging suspends recording and delivery; the unfamiliar role and missing ticket justify investigation in this synthetic scenario. [3]

A07 includes administrator, so the admin substring triggers the baseline. I labelled it close because the independent fixture confirms who made the change, where, what they did, and when; the word approved in the alert alone does not grant that exception. All twelve are invented scenarios, not employer logs. Per-alert references and full rationales are in SOURCES.md and my_alerts.json.

Results and operating point

Setting	Change	Caught /6	Missed /6	Over /6
K0	Lecture keywords	3	3	2
K1	K0 + credential	4	2	2
K2	K1 + outbound	5	1	3
K3	K2 + failed	5	1	4
K4	K3 + password	5	1	5
G1	K2 + context node/tool	6	0	1

K0 uses rce, ransomware, exfil, critical, and admin. Each keyword experiment adds exactly one word. G1 compares with K2, keeping its seven words fixed and adding the context-review node/tool. Its two policies form one combined intervention; this experiment does not isolate their individual causal effects.

The two errors and the people handling them

If I were staffing the team, I would accept some extra reviews to avoid missing an active incident, but I would not widen the rule without checking the workload. Adding credential catches A04 without another false alarm. Adding outbound catches A05 but also sends the routine A09 backup to an analyst. Adding failed and password catches nothing new here and adds A11 and A10 to the queue.

The setting I would choose

I would choose G1 for a supervised pilot. It catches all six labelled incidents and still sends A09 for unnecessary review, so that is the known error I am accepting. Removing outbound would avoid that backup review but miss the payroll transfer in A05; I would rather verify the backup than close that unexplained transfer.

G1 produces seven escalations: six labelled incidents and one routine event, compared with eight escalations under K2. If each false alarm took ten minutes, the routine review burden would fall from thirty to ten minutes per replay of these twelve examples. Those minutes are an illustrative staffing assumption, not measured analyst time or a daily-volume estimate.

What stayed wrong, and what the graph bought

A06 is missed and A07/A08 are over-escalated at every tested keyword setting. The text rule cannot check the asset role or verify a maintenance approval. Compared with K2, G1 catches one additional incident and removes two false alarms: A06 goes to investigate, while A07 and A08 go to quick_close. A09 stays wrong because there is no backup-job lookup.

The three scores and the IDs behind them are saved in results_mine.csv and evaluation_details.json. A single accuracy percentage would hide the trade between a missed incident and wasted review time, so I did not use one.

Evidence, comparison, and limits

LangGraph compared with my LangChain pipeline

My earlier LCEL workflow followed retrieval, prompt construction, and an answer stage. Here, traces.json records different executed paths: baseline A06 takes classify -> quick_close; G1 A06 takes classify -> context_review -> investigate. G1 A07 takes the same first two nodes but ends at quick_close. The conditional edge makes that branching explicit rather than forcing every alert through the same steps.

The memory output shows incident-A06 retaining its structured event when I submit only follow-up text. It remains escalated, incident-A07 remains unchanged, and an unused thread has no state. A plain stateless chain does not provide that checkpoint behavior on its own. LangChain can also branch and use external memory; LangGraph gives this project explicit state, routing, and checkpoints in one workflow. A fixed retrieval-and-answer task would still be simpler as a chain.

What the execution actually verifies

The completed project notebook ran all 14 code cells without errors, and 23 regression checks passed. They cover actual branches, approval mismatches, time boundaries, forged approval prose, label exclusion, repeatability, and thread isolation. The two lecture notebooks ran 5 and 6 code cells through their guarded no-key paths; the lecture URL verdict is a stub, not an LLM result.

The tested project environment used Python 3.12.3 and LangGraph 1.2.20. Input hashes and package versions are recorded in run_environment.json. The practice CVE identifier is a fictional course fixture, and the custom context tool reads only local simulated data.

Honest limits and next measurements

One example is one-sixth of either class, about 16.7 percentage points. I used the same authored examples to develop and compare the rules, so zero observed misses does not establish a low deployment miss rate. The graph also trusts structured event metadata and fixture records; stale, incomplete, or compromised context could reverse a decision, and missing inventory can leave a quiet incident closed.

Next I would freeze the policy and collect a separately labelled, time-separated sample with another analyst. I would measure missed incidents by severity, false alarms per day, review minutes, and results when inventory or change data is missing. A separate backup lookup and individual context-policy ablations would show whether the remaining error can be reduced without hiding incidents.

References

[1] LangChain. LangGraph Graph API.

[2] LangChain. Persistence.

[3] AWS. CloudTrail StopLogging API.

Additional MITRE ATT&CK and Microsoft event references are mapped to the individual scenarios in SOURCES.md. Sources checked November 10, 2026.